

University of Padova

Optimization for Data Science

Optimization Methods for the LASSO Problem

Group 9

Erdem Altun
Berke Kaan Dede
Andela Ruta Stancu

Academic Year 2025–2026

Abstract

This project investigates three first-order optimization methods for the constrained LASSO problem: Classic Frank-Wolfe, Pairwise Frank-Wolfe, and Projected Gradient. The methods are evaluated on three real-world regression datasets Riboflavin, YearPredictionMSD, and E2006, which represent substantially different dimensional and computational settings. The comparison focuses on objective reduction, Frank-Wolfe optimality gap, computational runtime, and sparsity of the resulting coefficient vectors. The experiments show that the behavior of the methods depends strongly on the structure of the dataset. Pairwise Frank-Wolfe provides the strongest overall optimization performance, obtaining the lowest final objective on Riboflavin and E2006 and being the only method to satisfy the prescribed 10^{-5} FW gap tolerance on YearPredictionMSD. Classic Frank-Wolfe maintains sparse feasible solutions but generally exhibits slower reduction of the optimality gap. Projected Gradient provides smooth objective reduction but may generate substantially denser intermediate and final solutions in high-dimensional problems. Overall, the experiments illustrate the trade-off between convergence, computational cost, and sparsity when solving constrained LASSO problems.

Contents

1	Introduction	4
2	The LASSO Problem	4
2.1	LASSO as a Constrained Least-Squares Problem	4
2.2	The ℓ_1 -Ball and Sparsity	5
3	Optimization Methods	5
3.1	Classic Frank-Wolfe	5
3.2	Pairwise Frank-Wolfe	8
3.3	Projected Gradient	10
4	Experimental Setup	15
4.1	Datasets	16
4.2	Data Preprocessing	17
4.3	Experimental Parameters	17
5	Numerical Results	18
5.1	Classic Frank-Wolfe Results	18
5.1.1	Riboflavin	19
5.1.2	YearPredictionMSD	19
5.1.3	E2006	19
5.2	Pairwise Frank-Wolfe Results	20
5.2.1	Riboflavin	20
5.2.2	YearPredictionMSD	20
5.2.3	E2006	21
5.3	Projected Gradient Results	21
5.3.1	Riboflavin	22
5.3.2	YearPredictionMSD	22
5.3.3	E2006	23
6	Comparison and Discussion	23
6.1	Dataset Results Comparison	23
6.2	Current Limitations and Future Developments	25
7	Conclusion	25
	Declaration on the Use of AI Tools	26

A	Pairwise Frank-Wolfe Convergence Plots	28
B	Classic Frank-Wolfe Convergence Plots	30
C	Projected Gradient Convergence Plots	32

1 Introduction

Sparse regression is widely used in statistical learning problems involving a large number of features. Including all available features may increase model complexity without necessarily improving prediction performance. Sparse models address this issue by using a smaller number of relevant features, which can also make the final model easier to interpret. One of the most commonly used techniques for obtaining sparse solutions is the Least Absolute Shrinkage and Selection Operator (LASSO). By imposing an ℓ_1 -norm constraint, LASSO encourages some coefficients to become exactly zero, allowing feature selection and coefficient estimation to be performed within the same model [2].

The constrained LASSO problem can be solved using different first-order optimization methods. Frank-Wolfe methods are useful in this setting because they avoid the need to project onto the feasible set at every iteration. Instead, they solve a linear minimization problem over the constraint set and use the resulting solution to update the current point [2, 5]. This approach can be particularly useful in large-scale problems, where projection onto the feasible set may be computationally expensive [4].

This project considers three optimization methods: Classic Frank-Wolfe, Pairwise Frank-Wolfe, and Projected Gradient. Classic Frank-Wolfe uses the standard Frank-Wolfe update direction, whereas the pairwise variant allows weight to be transferred between elements of the active representation [7]. Projected Gradient follows a different approach by first taking a gradient step and then projecting the resulting point back onto the feasible set. Therefore, the three methods handle the constraint in different ways during the optimization process.

The algorithms are evaluated on three real-world regression datasets: Riboflavin, YearPredictionMSD, and E2006. They differ considerably in size and structure, allowing the methods to be tested in different problem settings. The comparison considers objective function values, convergence behavior, computational runtime, and the sparsity of the final solutions. The aim is to investigate the practical advantages and limitations of each method rather than to focus only on the final objective value.

2 The LASSO Problem

2.1 LASSO as a Constrained Least-Squares Problem

In this project, the regression problem is formulated as a least-squares problem with an ℓ_1 -norm constraint on the coefficient vector. This constrained formulation corresponds to the LASSO problem and provides a common optimization problem for the three methods considered in the study. Following the formulation presented in [2, Eq. (12)], the problem considered in this project is

$$\begin{aligned} \min_{x \in \mathbb{R}^p} \quad & f(x) = \|Ax - b\|_2^2 \\ \text{subject to} \quad & \|x\|_1 \leq \tau. \end{aligned} \tag{1}$$

The matrix $A \in \mathbb{R}^{n \times p}$ contains the input features, where n denotes the number of observations and p denotes the number of features. The vector $b \in \mathbb{R}^n$ contains the observed target values, while $x \in \mathbb{R}^p$ is the coefficient vector to be estimated. The least-squares objective measures how closely the predictions Ax reproduce the observed targets.

The parameter τ determines how strongly the coefficient vector is restricted. When τ is small,

the feasible set is more restrictive and the model is encouraged to use fewer or smaller coefficients. A larger value of τ permits coefficient vectors with a larger ℓ_1 -norm. In this way, τ influences both the flexibility and the sparsity of the resulting model [2].

For the numerical experiments, the same constrained LASSO formulation is used for Classic Frank-Wolfe, Pairwise Frank-Wolfe, and Projected Gradient. This makes it possible to compare the behavior of the algorithms while keeping the underlying optimization problem unchanged.

2.2 The ℓ_1 -Ball and Sparsity

The constraint in problem (1) defines the feasible region as an ℓ_1 -ball. It can be written as

$$C = \{x \in \mathbb{R}^p : \|x\|_1 \leq \tau\}. \quad (2)$$

The same feasible region can also be represented as the convex hull of its extreme points [2, 4],

$$C = \text{conv} \{\pm \tau e_i : i = 1, \dots, p\}, \quad (3)$$

where e_i is the i -th standard basis vector in $\mathbb{R}^p$. Therefore, every extreme point of the feasible region has only one nonzero coordinate.

This geometry is closely related to sparsity. Instead of representing a feasible point as an arbitrary dense vector, points in the ℓ_1 -ball can be viewed as convex combinations of sparse extreme points. This property is particularly relevant for Frank-Wolfe algorithms, whose iterates are built from atoms selected from the feasible region. As a result, the structure of the constraint naturally supports sparse representations during the optimization process [2, 5].

The specific way in which Classic Frank-Wolfe selects these atoms through its linear minimization oracle is discussed in Section 3.1.

3 Optimization Methods

We use three first-order optimization methods for the constrained LASSO problem: Classic Frank-Wolfe, Pairwise Frank-Wolfe, and Projected Gradient. The two Frank-Wolfe methods avoid projection onto the feasible set and use a Linear Minimization Oracle (LMO) to select feasible points that define their search directions. Projected Gradient instead takes a gradient step and then projects the resulting point onto the constraint set. Pairwise Frank-Wolfe also works with the active representation of the current iterate and allows weight to be transferred between its elements. The following subsections describe each method and its implementation in the numerical experiments [2, 4, 5, 7].

3.1 Classic Frank-Wolfe

Classic Frank-Wolfe is a first-order method for constrained optimization that avoids explicit projection onto the feasible set. At each iteration, it solves a linear minimization problem using the gradient at the current iterate. The solution is obtained through a Linear Minimization Oracle (LMO) and is used to define the next search direction. Since the update is formed as a convex combination of the current iterate and the point returned by the LMO, feasibility is maintained during the iterations. For smooth convex problems, Classic Frank-Wolfe has the standard sublinear convergence rate of $\mathcal{O}(1/k)$ in terms of objective suboptimality [2, 5].

For the constrained least-squares problem considered in this project, the residual at iteration k is defined as

$$r_k = Ax_k - b. \quad (4)$$

Using this residual, the gradient of

$$f(x) = \|Ax - b\|_2^2 \quad (5)$$

can be written as

$$\nabla f(x_k) = 2A^T r_k. \quad (6)$$

This gradient is used in the Frank-Wolfe linear minimization subproblem,

$$s_k \in \arg \min_{s \in C} \langle \nabla f(x_k), s \rangle, \quad (7)$$

where the feasible region in this project is

$$C = \{x \in \mathbb{R}^p : \|x\|_1 \leq \tau\}. \quad (8)$$

For an ℓ_1 -ball, the linear minimization problem has a simple closed-form solution. The algorithm first finds a coordinate corresponding to the largest absolute component of the gradient,

$$j_k \in \arg \max_j |\nabla_j f(x_k)|, \quad (9)$$

and then constructs

$$s_k = -\tau \operatorname{sign}(\nabla_{j_k} f(x_k)) e_{j_k}. \quad (10)$$

The point returned by the LMO therefore has only one nonzero coordinate and is an extreme point of the ℓ_1 -ball. This makes the linear minimization step particularly simple for the constrained LASSO problem [2, 4].

Once s_k has been selected, the Frank-Wolfe search direction is

$$d_k = s_k - x_k. \quad (11)$$

The objective value at the current iterate is evaluated as

$$f(x_k) = \|Ax_k - b\|_2^2 = \|r_k\|_2^2. \quad (12)$$

The implementation also computes the Frank-Wolfe gap,

$$g_{\text{FW}}(x_k) = \langle x_k - s_k, \nabla f(x_k) \rangle. \quad (13)$$

For a convex objective, the Frank-Wolfe gap gives an upper bound on the objective suboptimality,

$$f(x_k) - f(x^*) \leq g_{\text{FW}}(x_k), \quad (14)$$

which makes it possible to assess convergence without knowing the exact optimal value $f(x^*)$. In the implementation, the algorithm stops when the FW gap is less than or equal to 10^{-5} . Otherwise, it continues until the maximum number of iterations for the dataset is reached [5].

If the stopping condition is not satisfied, the step size is determined using exact line search. The step is chosen by minimizing the objective along the current Frank-Wolfe direction over the interval $[0, 1]$,

$$\alpha_k \in \arg \min_{\alpha \in [0, 1]} f(x_k + \alpha d_k). \quad (15)$$

This exact line-search formulation is described in the Frank-Wolfe literature [5]. For the quadratic least-squares objective used here, this one-dimensional problem can be solved analytically. Using $r_k = Ax_k - b$, the objective along the direction becomes

$$f(x_k + \alpha d_k) = \|r_k + \alpha A d_k\|_2^2. \quad (16)$$

Since this expression is quadratic in α , the unconstrained minimizer is

$$\alpha_{\text{opt}} = -\frac{\langle r_k, A d_k \rangle}{\|A d_k\|_2^2}. \quad (17)$$

Here,

$$\|A d_k\|_2^2 = (A d_k)^T (A d_k), \quad (18)$$

which is the squared Euclidean norm of the transformed direction. The step size is then restricted to the valid Frank-Wolfe interval:

$$\alpha_k = \text{clip} \left(-\frac{\langle r_k, A d_k \rangle}{\|A d_k\|_2^2}, 0, 1 \right). \quad (19)$$

This closed-form expression is obtained by solving the exact line-search problem analytically for the quadratic least-squares objective used in this project.

Finally, the coefficient vector is updated according to

$$x_{k+1} = x_k + \alpha_k d_k. \quad (20)$$

Since $d_k = s_k - x_k$, the same update can also be written as

$$x_{k+1} = (1 - \alpha_k) x_k + \alpha_k s_k. \quad (21)$$

Because $\alpha_k \in [0, 1]$, the new iterate is a convex combination of two feasible points and therefore remains inside the ℓ_1 -ball. This gives the feasibility-preserving update used by Classic Frank-Wolfe [2].

Algorithm 1 Classic Frank-Wolfe for Constrained LASSO

Require: Feature matrix A , target vector b , radius τ , gap tolerance ε , maximum iterations K

Ensure: Solution vector x

```
1: Initialize  $x_0 \leftarrow 0$ 
2: for  $k = 0, 1, \dots, K$  do
3:   Compute residual  $r_k \leftarrow Ax_k - b$ 
4:   Compute gradient  $g_k \leftarrow \nabla f(x_k) = 2A^T r_k$ 
5:   Find  $j_k \leftarrow \arg \max_j |g_{k,j}|$ 
6:   Set  $s_k \leftarrow -\tau \operatorname{sign}(g_{k,j_k})e_{j_k}$ 
7:   Compute FW direction  $d_k \leftarrow s_k - x_k$ 
8:   Compute objective  $f(x_k) \leftarrow \|r_k\|_2^2$ 
9:   Compute FW gap  $g_{\text{FW}}(x_k) \leftarrow \langle x_k - s_k, g_k \rangle$ 
10:  if  $g_{\text{FW}}(x_k) \leq \varepsilon$  then
11:    break
12:  end if
13:  if  $k = K$  then
14:    break
15:  end if
16:  Compute  $q_k \leftarrow \|Ad_k\|_2^2$ 
17:  if  $q_k \leq 10^{-15}$  then
18:    break
19:  end if
20:  Compute exact line-search step  $\alpha_k \leftarrow \operatorname{clip}\left(-\frac{\langle r_k, Ad_k \rangle}{q_k}, 0, 1\right)$ 
21:  Update  $x_{k+1} \leftarrow x_k + \alpha_k d_k$ 
22: end for
23: return  $x$ 
```

3.2 Pairwise Frank-Wolfe

Standard Frank-Wolfe can exhibit a sublinear convergence rate of $\mathcal{O}(1/k)$ when the optimal solution lies on the boundary of the feasible polytope. In this setting, the method may suffer from a “zigzagging” phenomenon, repeatedly moving toward new vertices while being unable to directly reduce the weight of unfavorable atoms already present in the current representation [7].

Pairwise Frank-Wolfe (PFW) addresses this limitation by transferring weight directly from an active element v_k to a forward atom s_k . Instead of using the standard Frank-Wolfe direction $s_k - x_k$, the pairwise method uses the direction $s_k - v_k$ [7].

Active Representation and Atom Selection.

At iteration k , the current iterate is represented as a convex combination of elements in an active representation:

$$x_k = \sum_{v \in \mathcal{S}_k} \alpha_v v, \quad \sum_{v \in \mathcal{S}_k} \alpha_v = 1, \quad \alpha_v > 0. \quad (22)$$

For the ℓ_1 -ball

$$C = \{x \in \mathbb{R}^p : \|x\|_1 \leq \tau\}, \quad (23)$$

the non-origin atoms correspond to the extreme points $\pm \tau e_j$. In our implementation, the feasible

origin $x_0 = 0$ is additionally used as the initial element of the active representation.

At each iteration, PFW selects a forward atom and an away atom. The forward atom is obtained using the standard linear minimization oracle:

$$s_k \in \arg \min_{s \in C} \langle g_k, s \rangle, \quad (24)$$

where $g_k = \nabla f(x_k)$. For the ℓ_1 -ball, this can be computed by selecting

$$j_{\text{fw}} \in \arg \max_j |g_{k,j}|, \quad (25)$$

and setting

$$s_k = -\tau \operatorname{sign}(g_{k,j_{\text{fw}}}) e_{j_{\text{fw}}}. \quad (26)$$

The away atom is selected from the current active representation as

$$v_k \in \arg \max_{v \in S_k} \langle g_k, v \rangle. \quad (27)$$

When the origin is still present in the active representation, $\langle g_k, 0 \rangle = 0$ is used in this comparison.

Pairwise Direction and Exact Line Search.

The pairwise search direction is

$$d_k = s_k - v_k. \quad (28)$$

Moving along this direction transfers convex weight directly from v_k to s_k . To ensure that the weight of the away element does not become negative, the maximum admissible step size is

$$\alpha_{\max} = \alpha_{v_k}. \quad (29)$$

For the constrained least-squares objective used in this project, define the residual as

$$r_k = Ax_k - b. \quad (30)$$

Then

$$f(x_k + \alpha d_k) = \|r_k + \alpha A d_k\|_2^2. \quad (31)$$

The unconstrained minimizer along the pairwise direction is therefore

$$\alpha_{\text{opt}} = -\frac{\langle r_k, A d_k \rangle}{\|A d_k\|_2^2}. \quad (32)$$

The step size used by the implementation is the exact minimizer restricted to the feasible interval:

$$\alpha_k = \operatorname{clip} \left(-\frac{\langle r_k, A d_k \rangle}{\|A d_k\|_2^2}, 0, \alpha_{\max} \right). \quad (33)$$

State Updates and Pruning.

After computing α_k , the iterate and residual are updated according to

$$x_{k+1} = x_k + \alpha_k d_k, \quad r_{k+1} = r_k + \alpha_k A d_k. \quad (34)$$

The corresponding convex weights are updated by transferring mass from the away element to the forward atom:

$$\alpha_{s_k} \leftarrow \alpha_{s_k} + \alpha_k, \quad \alpha_{v_k} \leftarrow \alpha_{v_k} - \alpha_k. \quad (35)$$

If the updated weight of an active element becomes numerically zero, it is removed from the active representation. In the implementation, a threshold of 10^{-12} is used for this pruning step.

Algorithm 2 Pairwise Frank-Wolfe for Constrained LASSO

Require: Feature matrix A , target vector b , radius τ , gap tolerance ε , maximum iterations K

Ensure: Solution vector x

- 1: Initialize $x_0 \leftarrow 0$, $r_0 \leftarrow -b$
 - 2: Initialize the active representation with the origin having weight 1
 - 3: **for** $k = 0, 1, \dots, K - 1$ **do**
 - 4: Compute gradient $g_k \leftarrow 2A^T r_k$
 - 5: Find $j_{\text{fw}} \leftarrow \arg \max_j |g_{k,j}|$
 - 6: Set $s_k \leftarrow -\tau \text{sign}(g_{k,j_{\text{fw}}}) e_{j_{\text{fw}}}$
 - 7: Compute FW gap $g_{\text{FW}}(x_k) \leftarrow \langle g_k, x_k \rangle + \tau |g_{k,j_{\text{fw}}}|$
 - 8: **if** $g_{\text{FW}}(x_k) \leq \varepsilon$ **then**
 - 9: **break**
 - 10: **end if**
 - 11: Select $v_k \leftarrow \arg \max_{v \in \mathcal{S}_k} \langle g_k, v \rangle$
 - 12: Set pairwise direction $d_k \leftarrow s_k - v_k$
 - 13: Set $\alpha_{\text{max}} \leftarrow \alpha_{v_k}$
 - 14: Compute $\alpha_k \leftarrow \text{clip} \left(-\frac{\langle r_k, A d_k \rangle}{\|A d_k\|_2^2}, 0, \alpha_{\text{max}} \right)$
 - 15: Update $x_{k+1} \leftarrow x_k + \alpha_k d_k$
 - 16: Update $r_{k+1} \leftarrow r_k + \alpha_k A d_k$
 - 17: Increase the weight of s_k by α_k and decrease the weight of v_k by α_k
 - 18: Remove elements whose updated weight is at most 10^{-12}
 - 19: **end for**
 - 20: **return** x
-

3.3 Projected Gradient

Projected Gradient is a first-order method for constrained optimization that handles feasibility differently from the Frank-Wolfe methods considered in the previous subsections. While Frank-Wolfe avoids projections by solving a linear minimization problem over the feasible set, Projected Gradient first takes a gradient step and computes its projection onto the constraint set. In the implementation used in this project, the projected point is then used to define a feasible search direction before the final update. Projection-based methods therefore require the solution of a projection problem at every iteration, whereas Frank-Wolfe methods replace this operation with a Linear Minimization Oracle (LMO) [2, 4, 5].

For the constrained least-squares problem considered in this project, the residual at iteration k is defined as

$$r_k = Ax_k - b. \quad (36)$$

The objective function is

$$f(x_k) = \|Ax_k - b\|_2^2 = \|r_k\|_2^2. \quad (37)$$

Using the residual in (36), the gradient is given by

$$g_k = \nabla f(x_k) = 2A^T(Ax_k - b) = 2A^T r_k. \quad (38)$$

As in the other methods, the optimization is performed over the ℓ_1 -ball

$$C = \{x \in \mathbb{R}^p : \|x\|_1 \leq \tau\}. \quad (39)$$

The parameter τ therefore determines the radius of the feasible set and controls the amount of shrinkage imposed on the coefficient vector. Projection onto the ℓ_1 -ball can be reduced to a structured projection problem, and its computational requirements differ from the linear minimization operation used by Frank-Wolfe [2, 4].

In the implementation used in this project, the gradient step size is selected using an estimate of the Lipschitz constant of the gradient. Since the Hessian of the quadratic objective is $2A^T A$, the Lipschitz constant is

$$L = 2\|A\|_2^2, \quad (40)$$

where $\|A\|_2$ denotes the spectral norm of A . The implementation avoids a complete singular value decomposition and approximates the spectral norm by power iteration. Starting from a normalized vector $v^{(0)}$, the iteration is

$$v^{(t+1)} = \frac{A^T A v^{(t)}}{\|A^T A v^{(t)}\|_2}, \quad (41)$$

and after the prescribed number of power iterations the spectral norm is approximated by

$$\sigma \approx \|A v^{(M)}\|_2 \approx \|A\|_2. \quad (42)$$

The implementation uses $M = 20$ power iterations and then sets

$$L = 2\sigma^2 \quad (43)$$

and

$$\eta = \frac{1}{L}. \quad (44)$$

For the quadratic objective considered here, this choice provides a dataset-dependent scale for the gradient step.

Starting from the current feasible iterate x_k , an unconstrained gradient step is first computed as

$$y_k = x_k - \eta g_k. \quad (45)$$

The point y_k is not necessarily feasible. Therefore, it is projected onto the ℓ_1 -ball by solving the Euclidean projection problem

$$z_k = \Pi_C(y_k) = \arg \min_{z \in C} \frac{1}{2} \|z - y_k\|_2^2. \quad (46)$$

Projection explicitly enforces the constraint, so the resulting point satisfies

$$\|z_k\|_1 \leq \tau. \quad (47)$$

In the numerical implementation, projection onto the ℓ_1 -ball is computed using a sorting-based thresholding procedure. If the tentative point is already feasible, namely

$$\|y_k\|_1 \leq \tau, \quad (48)$$

then no correction is required and $z_k = y_k$. Otherwise, let

$$u_1 \geq u_2 \geq \dots \geq u_p \quad (49)$$

denote the components of $|y_k|$ sorted in decreasing order. The index used to determine the projection threshold is

$$\rho = \max \left\{ j \in \{1, \dots, p\} : u_j - \frac{\sum_{i=1}^j u_i - \tau}{j} > 0 \right\}. \quad (50)$$

The corresponding threshold is

$$\theta = \frac{\sum_{i=1}^{\rho} u_i - \tau}{\rho}. \quad (51)$$

The projected coordinates are then obtained according to

$$(z_k)_i = \text{sign}((y_k)_i) \max \{|(y_k)_i| - \theta, 0\}. \quad (52)$$

This thresholding operation ensures that the projected point belongs to the ℓ_1 -ball. The use of an explicit projection is the main structural difference between Projected Gradient and the projection-free Frank-Wolfe methods [2, 4].

After the projection, the Projected Gradient search direction used in the code is

$$d_k = z_k - x_k. \quad (53)$$

The implementation also computes the projected-gradient mapping

$$PG_{\eta}(x_k) = \frac{x_k - z_k}{\eta}, \quad (54)$$

whose norm is recorded as

$$\|PG_\eta(x_k)\|_2 = \frac{\|x_k - z_k\|_2}{\eta} = \frac{\|d_k\|_2}{\eta}. \quad (55)$$

A projected-gradient mapping close to zero indicates that an additional projected-gradient step would produce little change in the current iterate. Consequently, the quantity in (55) is recorded as a method-specific stationarity diagnostic in the numerical experiments.

For consistency with the Frank-Wolfe experiments, the implementation also computes the Frank-Wolfe gap as a common optimality certificate. For a general convex feasible set, this gap is

$$g_{\text{FW}}(x_k) = \max_{s \in C} \langle \nabla f(x_k), x_k - s \rangle. \quad (56)$$

For a convex objective, the Frank-Wolfe gap is nonnegative and provides an upper bound on objective suboptimality. It can therefore be used as an optimality certificate even when the iterate itself is generated by another optimization method [2, 5].

For the ℓ_1 -ball in (39), the linear minimizer used only for computing the certificate is located at the coordinate of largest absolute gradient. Consequently, the FW gap can be evaluated directly as

$$g_{\text{FW}}(x_k) = g_k^T x_k + \tau \|g_k\|_\infty. \quad (57)$$

The Frank-Wolfe gap is not used to construct the Projected Gradient direction. It is used only as the common stopping and comparison criterion. In particular, the stopping condition implemented in the experiments is

$$g_{\text{FW}}(x_k) \leq \varepsilon, \quad \varepsilon = 10^{-5}. \quad (58)$$

If condition (58) is not satisfied, the algorithm continues until the dataset-specific maximum iteration budget is reached. The use of the FW gap as a common certificate makes it possible to evaluate Projected Gradient, Classic Frank-Wolfe, and Pairwise Frank-Wolfe using the same optimality measure [5].

The implementation does not immediately replace x_k by the projected point z_k . Instead, after forming the direction (53), it performs an exact line search along the segment connecting the two feasible points. The step size is defined by

$$\gamma_k \in \arg \min_{\gamma \in [0,1]} f(x_k + \gamma d_k). \quad (59)$$

For the quadratic least-squares objective, using $r_k = Ax_k - b$, the objective along the direction becomes

$$f(x_k + \gamma d_k) = \|r_k + \gamma A d_k\|_2^2. \quad (60)$$

Using the relation $g_k = 2A^T r_k$, the exact minimizer along this direction can be written as

$$\gamma_{\text{opt}} = -\frac{g_k^T d_k}{2\|A d_k\|_2^2}. \quad (61)$$

The implementation restricts this value to the interval $[0, 1]$. It also protects against a numerically negligible denominator, giving

$$\gamma_k = \begin{cases} 0, & 2\|Ad_k\|_2^2 \leq 10^{-30}, \\ \text{clip}\left(-\frac{g_k^T d_k}{2\|Ad_k\|_2^2}, 0, 1\right), & \text{otherwise.} \end{cases} \quad (62)$$

For the quadratic least-squares objective used in this project, this one-dimensional line-search problem can be solved analytically using the expression above.

The coefficient vector is then updated according to

$$x_{k+1} = x_k + \gamma_k d_k. \quad (63)$$

Since $d_k = z_k - x_k$, the update can equivalently be written as

$$x_{k+1} = (1 - \gamma_k)x_k + \gamma_k z_k. \quad (64)$$

Both x_k and z_k belong to the convex feasible set C , and $\gamma_k \in [0, 1]$. Therefore, x_{k+1} remains feasible:

$$\|x_{k+1}\|_1 \leq \tau. \quad (65)$$

The implementation additionally performs a final projection if a very small violation of (65) occurs because of floating-point rounding. This operation acts only as a numerical safeguard and does not change the mathematical update described above.

The algorithm is initialized at

$$x_0 = 0, \quad (66)$$

which is feasible because $\|x_0\|_1 = 0 \leq \tau$. The resulting Projected Gradient procedure used in the experiments is summarized in Algorithm 3.

Algorithm 3 Projected Gradient for Constrained LASSO

Require: Feature matrix A , target vector b , radius τ , gap tolerance ε , maximum iterations K , power iterations M

Ensure: Solution vector x

```
1: Initialize  $x_0 \leftarrow 0$ 
2: Initialize residual  $r_0 \leftarrow -b$ 
3: Estimate  $\sigma \approx \|A\|_2$  by  $M$  power iterations
4: Set  $L \leftarrow 2\sigma^2$ 
5: Set  $\eta \leftarrow 1/L$ 
6: for  $k = 0, 1, \dots, K - 1$  do
7:   Compute gradient  $g_k \leftarrow 2A^T r_k$ 
8:   Compute objective  $f(x_k) \leftarrow \|r_k\|_2^2$ 
9:   Compute FW gap  $g_{\text{FW}}(x_k) \leftarrow g_k^T x_k + \tau \|g_k\|_\infty$ 
10:  Compute gradient step  $y_k \leftarrow x_k - \eta g_k$ 
11:  Project onto the  $\ell_1$ -ball  $z_k \leftarrow \Pi_C(y_k)$ 
12:  Compute Projected Gradient direction  $d_k \leftarrow z_k - x_k$ 
13:  Compute Projected Gradient norm  $\|PG_\eta(x_k)\|_2 \leftarrow \|d_k\|_2/\eta$ 
14:  if  $g_{\text{FW}}(x_k) \leq \varepsilon$  then
15:    break
16:  end if
17:  Compute  $q_k \leftarrow 2\|Ad_k\|_2^2$ 
18:  if  $q_k \leq 10^{-30}$  then
19:     $\gamma_k \leftarrow 0$ 
20:  else
21:    Compute exact line-search step  $\gamma_k \leftarrow \text{clip}\left(-\frac{g_k^T d_k}{q_k}, 0, 1\right)$ 
22:  end if
23:  Update  $x_{k+1} \leftarrow (1 - \gamma_k)x_k + \gamma_k z_k$ 
24:  if  $\|x_{k+1}\|_1 > \tau$  then
25:     $x_{k+1} \leftarrow \Pi_C(x_{k+1})$ 
26:    Recompute residual  $r_{k+1} \leftarrow Ax_{k+1} - b$ 
27:  else
28:    Update residual  $r_{k+1} \leftarrow r_k + \gamma_k Ad_k$ 
29:  end if
30: end for
31: Perform final feasibility check
32: return  $x$ 
```

4 Experimental Setup

The numerical experiments were performed on three real-world regression datasets that differ considerably in size and feature structure. For each dataset, the feature matrix $A \in \mathbb{R}^{n \times p}$ contains n observations and p features, while the corresponding target values are given by $b \in \mathbb{R}^n$.

For each dataset, the optimization algorithms estimate a coefficient vector $x \in \mathbb{R}^p$, which determines how the features are combined to form the linear predictions Ax . The objective is to obtain a coefficient vector that fits the observed target values well while satisfying the ℓ_1 -norm constraint introduced in problem (1). The experiments therefore evaluate both the least-squares error and the sparsity of the resulting coefficient vector.

Classic Frank-Wolfe, Pairwise Frank-Wolfe, and Projected Gradient are applied to the same

processed dataset and the same ℓ_1 -ball within each experiment. This provides a consistent setting for comparing the three methods in terms of objective value, convergence behavior, runtime, and sparsity.

4.1 Datasets

Three real-world regression datasets were used in the experiments: Riboflavin, YearPredictionMSD, and E2006. Together, they represent very different relationships between the number of observations and the number of features, making them useful for comparing the behavior of the algorithms under different problem settings.

Table 1: Main characteristics of the datasets used in the experiments.

Dataset	n	p	Target
Riboflavin	71	4,088	Log riboflavin production rate
YearPredictionMSD	515,345	90	Song release year
E2006	16,087	150,360	Log stock-return volatility

Riboflavin.

This is a biological dataset concerning the production of riboflavin (vitamin B2) by *Bacillus subtilis*, with $n = 71$ observations and $p = 4,088$ gene-expression features. The response variable is the log-transformed riboflavin production rate, while the features are logarithmic gene-expression levels [3]. With observations heavily outnumbered by features, Riboflavin represents a high-dimensional regression setting.

We loaded the dataset from OpenML using `fetch_openml` with the dataset name `riboflavin` and version 1. The feature matrix and target values were then converted into numerical arrays before preprocessing.

YearPredictionMSD.

Derived from the Million Song Dataset, this dataset is used to predict the release year of a song from numerical audio features. In contrast to Riboflavin, it contains $n = 515,345$ observations but only $p = 90$ features. These consist of 12 average-timbre features and 78 timbre-covariance features. The release years in the dataset range from 1922 to 2011 [1].

We obtained the raw data from the UCI Machine Learning Repository as a compressed ZIP file. After extracting `YearPredictionMSD.txt`, we loaded it using `pandas`, taking the first column as the target vector and the remaining columns as the feature matrix. Rather than splitting the data into training and test sets, we used the full dataset of 515,345 observations as a single optimization instance, since the purpose of the experiments was to compare optimization behavior on the same LASSO problem rather than predictive generalization.

E2006.

This dataset is derived from a collection of 10-K financial reports and has the largest number of features among the three datasets. The E2006-tfidf representation contains $n = 16,087$ training observations and $p = 150,360$ features. Its features combine TF-IDF representations of unigrams with stock-volatility information from the preceding 12 months [8]. The prediction target is the logarithm of stock-return volatility over the 12 months following the publication of the report [6].

We obtained the compressed `E2006.train.bz2` file from the LIBSVM repository, decompressed it, and read it using `load_svmlight_file`. Because of the very large number of features, we

kept the resulting matrix in sparse format to reduce memory requirements.

4.2 Data Preprocessing

Before running the optimization algorithms, we preprocessed each dataset to place the features on comparable scales and to center the target values. Classic Frank-Wolfe, Pairwise Frank-Wolfe, and Projected Gradient were then applied to the same preprocessed data.

For Riboflavin and YearPredictionMSD, we standardized the feature matrices using `StandardScaler`, which centers each feature around zero and scales it by its standard deviation. The target vector was centered separately by subtracting its mean. Both steps were applied immediately after loading the two datasets.

E2006 was treated differently because its feature matrix was stored in sparse format. We standardized the features using `StandardScaler(with_mean=False)`, which scales them without explicitly centering the sparse matrix. This avoided converting the matrix into a dense representation, which would require substantially more memory given the large number of features. As with the other datasets, we centered the target vector by subtracting its mean.

No additional transformations were applied for the individual optimization methods. Each algorithm therefore received the same feature matrix and target vector for a given dataset, allowing the numerical results to be compared under the same preprocessing conditions.

4.3 Experimental Parameters

The radius τ of the ℓ_1 -ball was selected separately for each dataset based on preliminary experiments with several candidate values. The resulting ℓ_1 -norms were also considered as a practical indicator of whether the constraint provided a suitable feasible region for each problem. The aim was to avoid values that made the constraint unnecessarily restrictive or excessively loose.

In the illustrative LASSO experiment of Lacoste-Julien and Jaggi [7], the feasible region is chosen as a scaled ℓ_1 -ball with radius 20. However, this value is specific to their synthetic problem instance, which is generated using a random Gaussian matrix and a sparse artificial solution, rather than being a general rule for selecting τ . Since the present experiments use three real-world datasets with substantially different dimensions and feature structures, τ was instead selected separately for each dataset through preliminary experiments.

Based on these experiments, the final values were $\tau = 3$ for Riboflavin, $\tau = 10$ for YearPredictionMSD, and $\tau = 3$ for E2006. For each dataset, the same value of τ was then used for all three optimization methods so that the algorithms were compared over the same feasible region.

Table 2: Main experimental parameters for each dataset.

Dataset	τ	Maximum iterations
Riboflavin	3	5000
YearPredictionMSD	10	3000
E2006	3	3000

The maximum iteration budget was also fixed separately for each dataset and kept the same across the three methods. Riboflavin was assigned a maximum of 5000 iterations, while YearPredictionMSD and E2006 were assigned a maximum of 3000 iterations. These values represent

upper limits, since any of the three methods may terminate earlier when the stopping condition is satisfied.

All three methods use a Frank-Wolfe gap tolerance of 10^{-5} . The methods terminate when the gap is less than or equal to this tolerance or when the maximum iteration budget is reached. For Projected Gradient, the projected-gradient mapping norm is also recorded as an additional stationarity diagnostic, but it is not used as the stopping criterion.

All three methods start from the zero vector $x_0 = 0$, which is feasible because

$$\|x_0\|_1 = 0 \leq \tau. \quad (67)$$

This gives all three algorithms the same feasible starting point.

The step-size rule depends on the optimization method. Classic Frank-Wolfe uses exact line search along the Frank-Wolfe direction. Since the least-squares objective is quadratic, the exact minimizer is computed analytically and restricted to the interval $[0, 1]$. Pairwise Frank-Wolfe similarly uses the exact minimizer of the quadratic objective along the pairwise direction, restricted to the maximum admissible step determined by the weights in the active representation. For Projected Gradient, the gradient-step parameter is

$$\eta = \frac{1}{L}, \quad L = 2\|A\|_2^2, \quad (68)$$

where the spectral norm of A is estimated numerically for each dataset. This parameter is used to compute the gradient step before projection onto the ℓ_1 -ball. After the projection, the method forms the direction from the current iterate to the projected point and uses exact line search along this direction, with the resulting step restricted to the interval $[0, 1]$.

For the reported sparsity results, a coefficient was counted as nonzero when its absolute value was greater than 10^{-8} .

5 Numerical Results

5.1 Classic Frank-Wolfe Results

Classic Frank-Wolfe was evaluated on the three datasets using the dataset-specific values of τ and the maximum iteration budgets defined in Section 4.3. None of the three runs satisfied the FW gap tolerance of 10^{-5} , so each run terminated after reaching its maximum iteration budget. The final results are summarized in Table 3.

Table 3: Final Classic Frank-Wolfe results on the three datasets.

Dataset	Objective	FW gap	Iter.	Runtime (s)	$\ x\ _1$	Nonzero
Riboflavin	0.078025	0.142384	5000	1.06	2.9036	104
YearPredictionMSD	51276273.330381	24214.569200	3000	152.11	9.9707	20
E2006	1734.231804	225.718020	3000	275.11	2.6092	380

5.1.1 Riboflavin

For Riboflavin, the objective value decreased throughout the run. At iteration 1000, the objective was 0.222485, and it reached 0.078025 at iteration 5000. The FW gap also showed an overall decrease, from 0.586131 at iteration 1000 to 0.142384 at the final iteration. The objective and FW gap were still decreasing during the later iterations, indicating that the method had not yet reached the prescribed stopping tolerance when the iteration budget was exhausted. The total recorded runtime was approximately 1.06 seconds.

The final coefficient vector had an ℓ_1 -norm of approximately 2.9036, which remained within the feasible radius $\tau = 3$ and was relatively close to the boundary of the feasible set. The solution contained 104 nonzero coefficients out of 4,088 features. The number of nonzero coefficients increased rapidly during the early iterations and then largely stabilized, ending at 104 coefficients. Therefore, the final iterate remained feasible while using only a small subset of the available coefficients. The FW gap remained above the stopping tolerance at the end of the 5000 iterations.

5.1.2 YearPredictionMSD

For YearPredictionMSD, the objective decreased gradually during the 3000-iteration run. It was 51347098.794149 at iteration 500 and reached 51276273.330381 at the final iteration.

The FW gap did not decrease monotonically at every recorded point. In particular, it increased from 47405.512949 at iteration 1500 to 57459.621450 at iteration 2000, while the objective continued to decrease. Overall, the gap fell from 169812.186389 at iteration 500 to 24214.569200 at iteration 3000. Despite the fluctuations in the FW gap, the objective continued to decrease throughout the recorded iterations. The total recorded runtime was approximately 152.11 seconds.

The final solution had an ℓ_1 -norm of approximately 9.9707, remaining inside the ℓ_1 -ball with radius $\tau = 10$ and very close to its boundary. The coefficient vector contained 20 nonzero coefficients out of the 90 available features. The number of nonzero coefficients increased mainly during the early part of the run and then stabilized at 20 for the remaining iterations. Although the objective had decreased substantially, the final FW gap was still far above the prescribed tolerance, so the run ended because the maximum iteration budget was reached rather than because the stopping condition was satisfied.

5.1.3 E2006

For E2006, the objective value decreased from 2032.498017 at iteration 500 to 1734.231804 at iteration 3000. The FW gap showed some fluctuations during the run. It was 2963.120612 at iteration 500 and decreased to 583.507230 at iteration 1000, before increasing to 1188.157967 at iteration 1500. It then decreased again and reached 225.718020 at the final iteration. The objective continued to decrease throughout the recorded iterations, while the FW gap remained above the stopping tolerance. The total recorded runtime was approximately 275.11 seconds.

The final coefficient vector had an ℓ_1 -norm of approximately 2.6092, which satisfied the constraint with $\tau = 3$ and remained below the boundary of the feasible ball. The solution contained 380 nonzero coefficients out of 150,360 available features. This corresponds to approximately 0.25% of the available features, so the final coefficient vector remained sparse relative to the very high dimensionality of the dataset. The FW gap remained above 10^{-5} when the maximum budget of 3000 iterations was reached.

Overall, Classic Frank-Wolfe maintained feasible and sparse solutions on all three datasets. None

of the runs reached the FW gap tolerance of 10^{-5} within the available iteration budgets. The objective value decreased in all three experiments, while the FW gap showed an overall decreasing trend but was not necessarily monotone at every recorded iteration. The final ℓ_1 -norms also remained within the corresponding feasible radii in all three cases, consistent with the feasibility-preserving Frank-Wolfe update.

5.2 Pairwise Frank-Wolfe Results

This subsection presents the numerical performance and convergence behavior of Pairwise Frank-Wolfe on the three real-world datasets considered in the experiments.

5.2.1 Riboflavin

For the Riboflavin dataset ($n = 71$, $p = 4,088$), Pairwise Frank-Wolfe was run with $\tau = 3$ and a maximum budget of 5000 iterations.

The objective value decreased to 0.069040 by iteration 1000 and reached 0.026222 at iteration 5000. Over the same period, the FW gap decreased from 0.226459 at iteration 1000 to 0.034319 at the final iteration. Although both quantities decreased substantially, the FW gap tolerance of 10^{-5} was not reached within the maximum iteration budget.

The final ℓ_1 -norm was approximately 3.0, placing the solution on the boundary of the feasible ℓ_1 -ball up to floating-point precision. The final coefficient vector contained 83 nonzero coefficients out of 4,088 features.

Table 4: Final Pairwise Frank-Wolfe results on Riboflavin.

Metric	Value
Final objective	0.026222
Final FW gap	0.034319
Iterations	5000
Runtime (s)	0.6742
Final $\ x\ _1$	3.0000
Nonzero coefficients	83

5.2.2 YearPredictionMSD

For YearPredictionMSD ($n = 515,345$, $p = 90$), Pairwise Frank-Wolfe was run with $\tau = 10$, a FW gap tolerance of 10^{-5} , and a maximum budget of 3000 iterations.

The algorithm satisfied the stopping criterion at iteration 263, with a final FW gap of approximately 9.0413×10^{-6} . The corresponding objective value was 51260896.556391.

The final ℓ_1 -norm was approximately 10.0, and the final coefficient vector contained 20 nonzero coefficients out of the 90 available features.

Table 5: Final Pairwise Frank-Wolfe results on YearPredictionMSD.

Metric	Value
Final objective	51260896.556391
Final FW gap	9.0413×10^{-6}
Iterations	263
Runtime (s)	4.5518
Final $\ x\ _1$	10.0000
Nonzero coefficients	20

5.2.3 E2006

For E2006 ($n = 16,087$, $p = 150,360$), Pairwise Frank-Wolfe was run with $\tau = 3$ and a maximum budget of 3000 iterations.

The objective value decreased from 1712.865057 at iteration 500 to 1611.946492 at iteration 3000. Over the same recorded iterations, the FW gap decreased from 682.597918 to 10.669547. The algorithm reached the maximum iteration budget without satisfying the FW gap tolerance.

The final ℓ_1 -norm was approximately 3.0, placing the final solution on the boundary of the feasible ball up to floating-point precision. The final coefficient vector contained 659 nonzero coefficients out of 150,360 features.

Table 6: Final Pairwise Frank-Wolfe results on E2006.

Metric	Value
Final objective	1611.946492
Final FW gap	10.669547
Iterations	3000
Runtime (s)	342.2302
Final $\ x\ _1$	3.0000
Nonzero coefficients	659

5.3 Projected Gradient Results

Projected Gradient was evaluated on the three datasets using the dataset-specific values of τ and the maximum iteration budgets defined in Section 4.3. The stopping criterion was the same common FW gap tolerance used for the Frank-Wolfe experiments, namely 10^{-5} . None of the three Projected Gradient runs satisfied this tolerance, and therefore each run terminated after reaching its maximum iteration budget. The projected-gradient mapping norm was recorded separately as a method-specific stationarity diagnostic, but it was not used as the stopping criterion. The final results are summarized in Table 7.

The reported runtime includes both the spectral-norm estimation required to construct the step size $\eta = 1/L$ and the subsequent optimization iterations. The objective values should be interpreted separately for each dataset because the three regression problems differ considerably in sample size, dimensionality, and numerical scale.

Table 7: Final Projected Gradient results on the three datasets.

Dataset	Obj.	FW gap	PG norm	Iter.	Time (s)	$\ x\ _1$	NZ
Riboflavin	0.431275	2.452329	3.576219	5000	2.09	3.0000	352
YearPredictionMSD	51260896.556391	1.096945	0.152305	3000	146.33	10.0000	20
E2006	1706.564597	3366.201359	1477.811190	3000	257.81	3.0000	1694

5.3.1 Riboflavin

For the Riboflavin dataset ($n = 71$, $p = 4,088$), Projected Gradient was run with $\tau = 3$, a FW gap tolerance of 10^{-5} , and a maximum budget of 5000 iterations. The objective decreased rapidly during the first part of the run and then continued decreasing more gradually, reaching a final value of 0.431275. The FW gap followed the same general decreasing trend and reached 2.452329 at the final iteration.

The final projected-gradient mapping norm was 3.576219. Although both the objective function and the FW gap decreased substantially during the run, the final FW gap remained above the stopping tolerance of 10^{-5} . Consequently, the stopping criterion in (58) was not satisfied and the algorithm terminated after reaching the maximum budget of 5000 iterations.

The sparsity pattern changed considerably during the optimization process. A large number of coefficients became active during the early iterations, after which the number of nonzero coefficients decreased progressively. The final coefficient vector contained 352 nonzero coefficients out of the 4,088 available features.

The final ℓ_1 -norm was approximately 3.0, which places the solution on the boundary of the feasible ℓ_1 -ball with radius $\tau = 3$, up to floating-point precision. The measured feasibility violation was approximately 8.88×10^{-16} , which is numerical round-off rather than a meaningful violation of the constraint. The final solution can therefore be regarded as feasible up to floating-point precision.

5.3.2 YearPredictionMSD

For YearPredictionMSD ($n = 515,345$, $p = 90$), Projected Gradient was run with $\tau = 10$, a FW gap tolerance of 10^{-5} , and a maximum budget of 3000 iterations. The objective value decreased very rapidly during the early iterations before approaching a nearly constant value. The final objective was 51260896.556391.

The FW gap decreased by several orders of magnitude during the optimization process and reached a final value of 1.096945. Its convergence curve became nearly flat during the later part of the run. The final projected-gradient mapping norm was 0.152305.

Despite this decrease, the FW gap remained above the prescribed tolerance of 10^{-5} . The run therefore did not satisfy the formal stopping condition in (58) and terminated at the maximum budget of 3000 iterations. The total recorded runtime was approximately 146.33 seconds.

The sparsity pattern also stabilized rapidly. The final coefficient vector contained only 20 nonzero coefficients out of the 90 available features. The final ℓ_1 -norm was approximately 10.0, equal to the radius $\tau = 10$ up to floating-point precision. The corresponding feasibility violation was approximately 3.55×10^{-15} and is attributable to floating-point rounding. Thus, the final solution can be regarded as feasible and located on the boundary of the ℓ_1 -ball.

5.3.3 E2006

For E2006 ($n = 16,087$, $p = 150,360$), Projected Gradient was run with $\tau = 3$, a FW gap tolerance of 10^{-5} , and a maximum budget of 3000 iterations. The objective decreased throughout the run and reached 1706.564597 at the final iteration. The objective was still decreasing when the maximum iteration budget was reached.

The FW gap also showed an overall decrease during the run and reached a final value of 3366.201359. The corresponding projected-gradient mapping norm was 1477.811190. Both values remained relatively large compared with the prescribed FW gap tolerance of 10^{-5} . The total recorded runtime was approximately 257.81 seconds.

A substantial reduction in the number of active coefficients was observed during the iterations. Because E2006 contains 150,360 features, a large number of coefficients became active during the initial stage of the Projected Gradient procedure. This number subsequently decreased during the optimization process, leaving 1694 nonzero coefficients in the final solution.

The final coefficient vector had an ℓ_1 -norm of approximately 3.0, placing the solution on the boundary of the feasible set with $\tau = 3$, up to floating-point precision. The measured feasibility violation was approximately 2.22×10^{-15} , which is negligible and results from numerical round-off. The FW gap nevertheless remained substantially above 10^{-5} when the maximum budget was reached, so the E2006 run did not satisfy the formal stopping criterion.

6 Comparison and Discussion

The experiments show substantial differences between the three optimization methods even though they solve exactly the same constrained LASSO problem and use the same feasible region for each dataset. The results should therefore be interpreted jointly in terms of objective value, FW gap, runtime, and sparsity, rather than by considering a single metric. The convergence and sparsity trajectories supporting this comparison are reported in Appendices A, B, and C.

6.1 Dataset Results Comparison

For **Riboflavin**, Pairwise Frank-Wolfe showed the strongest overall performance among the three methods. It obtained the lowest final objective, approximately 0.0262, compared with 0.0780 for Classic Frank-Wolfe and 0.4313 for Projected Gradient. The same ordering can be observed in the FW gap: Pairwise Frank-Wolfe reached 0.0343, Classic Frank-Wolfe reached 0.1424, and Projected Gradient remained considerably higher at 2.4523. None of the three methods satisfied the prescribed tolerance of 10^{-5} , and all therefore used the full budget of 5000 iterations. Nevertheless, Pairwise Frank-Wolfe was also the fastest method, requiring only about 0.67 seconds, compared with 1.06 seconds for Classic Frank-Wolfe and 2.09 seconds for Projected Gradient. The final ℓ_1 -norm was approximately 3.00 for Pairwise Frank-Wolfe and Projected Gradient, placing both solutions on the boundary of the feasible ball, while Classic Frank-Wolfe remained slightly inside it with $\|x\|_1 \approx 2.90$. The sparsity results further distinguish the methods: Pairwise Frank-Wolfe finished with 83 nonzero coefficients, Classic Frank-Wolfe with 104, and Projected Gradient with 352. As shown in Figures A.1, B.1, and C.1, the Pairwise method initially increased its active support but was later able to remove some coefficients by transferring weight away from less useful atoms. In contrast, Projected Gradient activated a large number of coefficients during the early iterations and gradually reduced them through projection. Its final projected-gradient mapping norm was approximately 3.58, which, together with the remaining FW gap, indicates that the method was still away from stationarity when

the iteration budget was reached.

For **YearPredictionMSD**, the difference in convergence behavior is even more pronounced. Pairwise Frank-Wolfe was the only method that satisfied the common FW gap tolerance of 10^{-5} . It terminated after only 263 iterations with a final gap of approximately 9.04×10^{-6} and a runtime of about 4.55 seconds. Pairwise Frank-Wolfe and Projected Gradient reached essentially the same reported objective value, approximately 5.1261×10^7 . However, their convergence certificates were very different: the FW gap of Projected Gradient remained around 1.10, while Pairwise Frank-Wolfe reduced it below the required tolerance. Projected Gradient therefore continued until the maximum budget of 3000 iterations and required approximately 146.33 seconds. Classic Frank-Wolfe also used all 3000 iterations, with a slightly higher final objective of approximately 5.1276×10^7 , a much larger FW gap of approximately 2.42×10^4 , and a runtime of about 152.11 seconds. All three methods produced 20 nonzero coefficients. Their final ℓ_1 -norms were also very similar: approximately 10.00 for Pairwise Frank-Wolfe and Projected Gradient, and 9.97 for Classic Frank-Wolfe. The Projected Gradient mapping norm decreased to approximately 0.152, indicating that its projected steps had become small, but its FW certificate was still above the required tolerance. The graphs in Figures A.2, B.2, and C.2 therefore show an important point: a nearly constant objective value does not by itself imply that the formal optimality condition has been satisfied. In this dataset, Pairwise Frank-Wolfe reached essentially the same objective and sparsity level as Projected Gradient, but did so with a much stronger optimality certificate and substantially lower computational cost.

The **E2006** results show a different trade-off because this dataset has 150,360 features and therefore represents the most demanding high-dimensional setting. Pairwise Frank-Wolfe again achieved the lowest final objective, approximately 1611.95, followed by Projected Gradient at 1706.56 and Classic Frank-Wolfe at 1734.23. Pairwise Frank-Wolfe also obtained the smallest FW gap, approximately 10.67. Classic Frank-Wolfe finished with a gap of approximately 225.72, whereas Projected Gradient remained much further from the common optimality criterion, with a final FW gap of approximately 3.37×10^3 . Its projected-gradient mapping norm was also relatively large, approximately 1.48×10^3 . None of the methods reached the 10^{-5} stopping tolerance, so all three used the full budget of 3000 iterations. In terms of runtime, Projected Gradient was the fastest at approximately 257.81 seconds, followed by Classic Frank-Wolfe at 275.11 seconds, while Pairwise Frank-Wolfe required the most time, approximately 342.23 seconds. The sparsity comparison gives a different ordering: Classic Frank-Wolfe produced the sparsest final solution with only 380 nonzero coefficients, compared with 659 for Pairwise Frank-Wolfe and 1694 for Projected Gradient. Pairwise Frank-Wolfe and Projected Gradient both finished on the boundary of the constraint with $\|x\|_1 \approx 3.00$, whereas Classic Frank-Wolfe remained further inside the feasible region with $\|x\|_1 \approx 2.61$. As illustrated in Figures A.3, B.3, and C.3, this dataset therefore reveals a clear trade-off between the three methods. Pairwise Frank-Wolfe achieved the best objective and the strongest optimality certificate, but at the highest computational cost. Classic Frank-Wolfe produced the sparsest solution, while Projected Gradient was slightly faster and achieved a lower objective than Classic Frank-Wolfe, but retained substantially more coefficients and a considerably larger optimality gap.

Overall, Pairwise Frank-Wolfe showed the strongest convergence behavior across the experiments. Its pairwise direction allows weight to be transferred directly from an unfavorable active atom to a more favorable one, which can reduce the limitations of the standard Frank-Wolfe update [7]. Classic Frank-Wolfe remains attractive because its iterations are simple, projection-free, and naturally construct sparse solutions, although the experiments show that its FW gap can decrease slowly, particularly on YearPredictionMSD and E2006 [2, 5]. Projected Gradient follows a fundamentally different strategy: many coordinates may change simultaneously before projection onto the ℓ_1 -ball. This explains the very different sparsity trajectories observed for Riboflavin and E2006 and illustrates why projection cost becomes relevant in high-dimensional settings [4].

6.2 Current Limitations and Future Developments

Several limitations should be considered when interpreting these results and provide directions for future development. First, the experiments use fixed values of τ selected from preliminary tests and a fixed maximum iteration budget; a more extensive study could select τ through systematic cross-validation and evaluate sensitivity to this parameter. Second, the 10^{-5} FW gap tolerance is deliberately strict and was not reached by most runs, so larger computational budgets could provide a more complete comparison of asymptotic behavior. The Projected Gradient implementation could also be improved through adaptive Lipschitz estimation or backtracking strategies, while large scale Frank-Wolfe implementations could benefit from more efficient active-set management and screening techniques. Finally, the present study focuses on optimization performance on fixed regression instances rather than predictive generalization. Future work could combine the optimization analysis with train-test evaluation, repeated experiments, and larger or additional datasets to determine how the observed convergence and sparsity differences translate into predictive performance.

7 Conclusion

This project investigated Classic Frank-Wolfe, Pairwise Frank-Wolfe, and Projected Gradient for solving the constrained LASSO problem

$$\min_x \|Ax - b\|_2^2 \quad \text{subject to} \quad \|x\|_1 \leq \tau.$$

The three methods were implemented under a common experimental framework and evaluated on Riboflavin, YearPredictionMSD, and E2006. These datasets provided three substantially different optimization settings: a biological problem with many more features than observations, a very large dataset with relatively few features, and an extremely high-dimensional sparse text-regression problem. Using the same data preprocessing, feasible region, starting point, and FW gap tolerance for each dataset allowed the numerical behavior of the methods to be compared consistently.

The experiments demonstrate that no single performance measure is sufficient to characterize an optimization method. Objective reduction measures the quality of the current least-squares solution, while the FW gap provides a common certificate of optimality, runtime measures computational cost, and the number of nonzero coefficients describes the sparsity of the resulting model. The convergence plots further showed that an apparently stable objective does not necessarily imply that the prescribed optimality tolerance has been reached, as observed particularly for Projected Gradient on YearPredictionMSD.

Overall, the three methods showed different strengths depending on the dataset and the performance measure considered.

Pairwise Frank-Wolfe provided the strongest overall optimization performance. It achieved the lowest final objective on Riboflavin and E2006 and reached essentially the same objective value as Projected Gradient on YearPredictionMSD. It was also the only method that satisfied the common FW gap tolerance of 10^{-5} , reaching this criterion on YearPredictionMSD after only 263 iterations. Another advantage of the method is its pairwise update, which can transfer weight away from previously selected atoms. This was visible in the Riboflavin sparsity trajectory, where the number of active coefficients increased initially and then decreased during later iterations.

Classic Frank-Wolfe showed a different advantage. Its projection-free update is simple and naturally maintains sparse iterates. This was particularly clear on E2006, where it produced the

sparsest final solution among the three methods. However, its convergence was generally slower in terms of the FW gap. None of the Classic Frank-Wolfe runs reached the prescribed stopping tolerance within the available iteration budgets, even though the objective continued to decrease.

Projected Gradient achieved substantial objective reduction on all three datasets, but its behavior differed from the Frank-Wolfe methods because it updates the solution through an explicit projection onto the ℓ_1 -ball. This often produced denser intermediate solutions, especially on Riboflavin and E2006. On YearPredictionMSD, the objective stabilized close to the value obtained by Pairwise Frank-Wolfe, but the FW gap remained above the required tolerance. The projected gradient mapping norm provided an additional method-specific measure of stationarity, while the FW gap was kept as the common certificate for comparing all three methods.

Overall, the experiments support the practical motivation for using Frank-Wolfe variants in constrained sparse regression. Pairwise Frank-Wolfe was the most effective method in terms of optimization accuracy in the tested instances, while Classic Frank-Wolfe offered a favorable sparsity structure and Projected Gradient provided a useful projection-based baseline. The results therefore illustrate that the choice of optimization algorithm should consider not only the final objective value but also convergence guarantees, computational cost, sparsity requirements, and the dimensional structure of the underlying problem.

Declaration on the Use of AI Tools

Generative AI tools were used as a supporting aid to better understand how the mathematical formulations presented in the literature could be translated into computational steps. They were also used to check the consistency between the theoretical description and the implementation. The final code, experiments, results, and interpretation were developed, reviewed, and verified by the authors.

References

- [1] Thierry Bertin-Mahieux. Year prediction msd. UCI Machine Learning Repository, 2011.
- [2] Immanuel M. Bomze, Francesco Rinaldi, and Damiano Zeffiro. Frank-Wolfe and friends: A journey into projection-free first-order optimization methods. *4OR*, 19:313–345, 2021.
- [3] Peter Bühlmann, Markus Kalisch, and Lukas Meier. High-dimensional statistics with a view toward applications in biology. *Annual Review of Statistics and Its Application*, 1:255–278, 2014.
- [4] Cyrille W. Combettes and Sebastian Pokutta. Complexity of linear minimization and projection on some sets. arXiv:2101.10040v2, 2021.
- [5] Martin Jaggi. Revisiting Frank-Wolfe: Projection-free sparse convex optimization. In *Proceedings of the 30th International Conference on Machine Learning*, volume 28, 2013.
- [6] Shimon Kogan, Dmitry Levin, Bryan R. Routledge, Jacob S. Sagi, and Noah A. Smith. Predicting risk from financial reports with regression. In *Proceedings of Human Language Technologies: The 2009 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, pages 272–280, 2009.
- [7] Simon Lacoste-Julien and Martin Jaggi. On the global linear convergence of Frank-Wolfe optimization variants. arXiv:1511.05932v1, 2015.
- [8] LIBSVM. Libsvm data: Regression. LIBSVM Data Sets. E2006-tfidf regression dataset.

A Pairwise Frank-Wolfe Convergence Plots

The following figures report the objective value, Frank-Wolfe gap against iterations and CPU time, and sparsity evolution for Pairwise Frank-Wolfe on the three datasets.

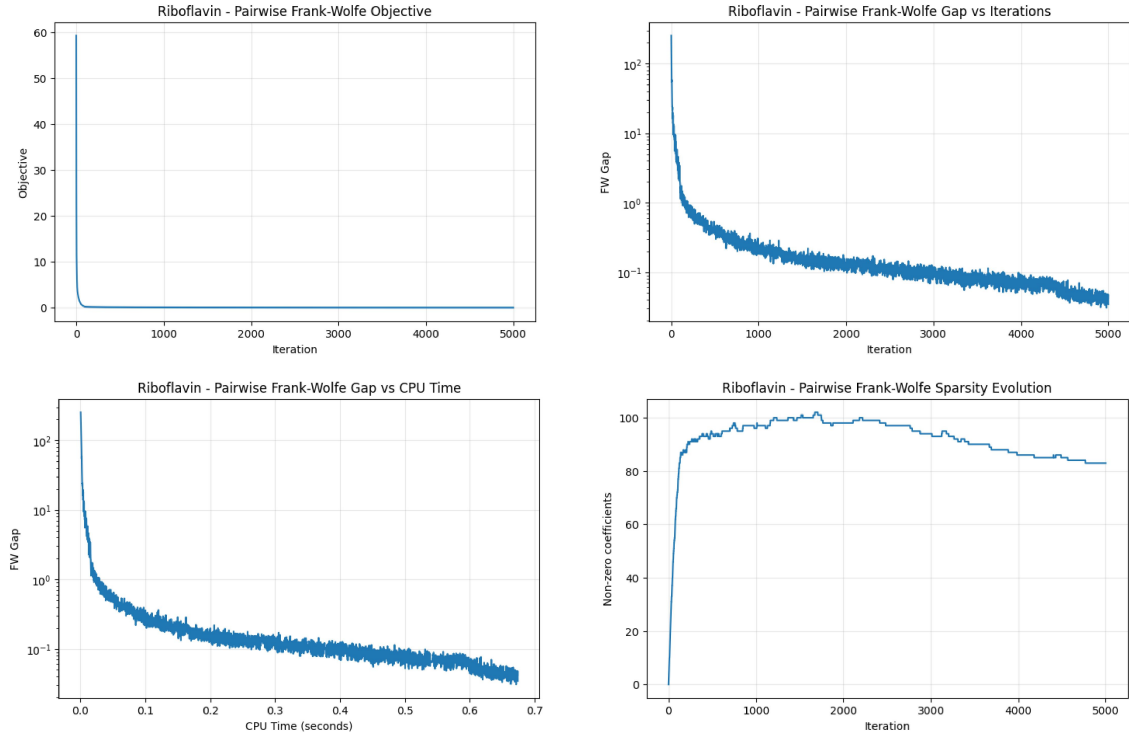


Figure A.1: Pairwise Frank-Wolfe convergence and sparsity results for Riboflavin.

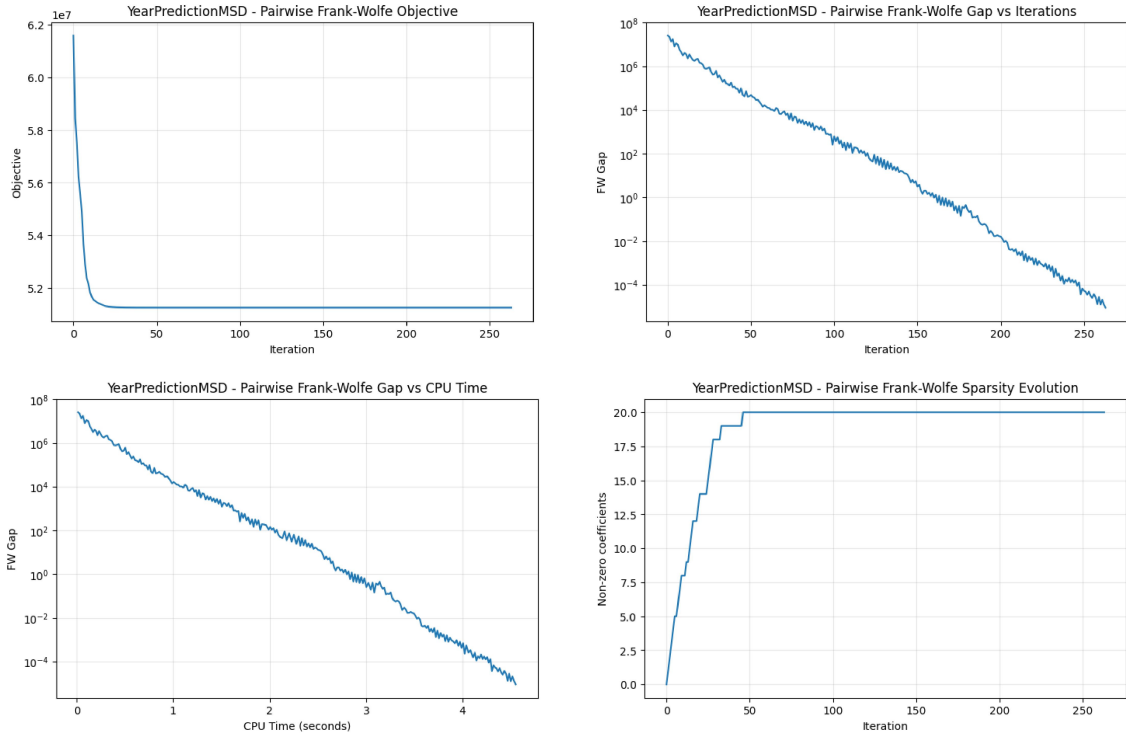


Figure A.2: Pairwise Frank-Wolfe convergence and sparsity results for YearPredictionMSD.

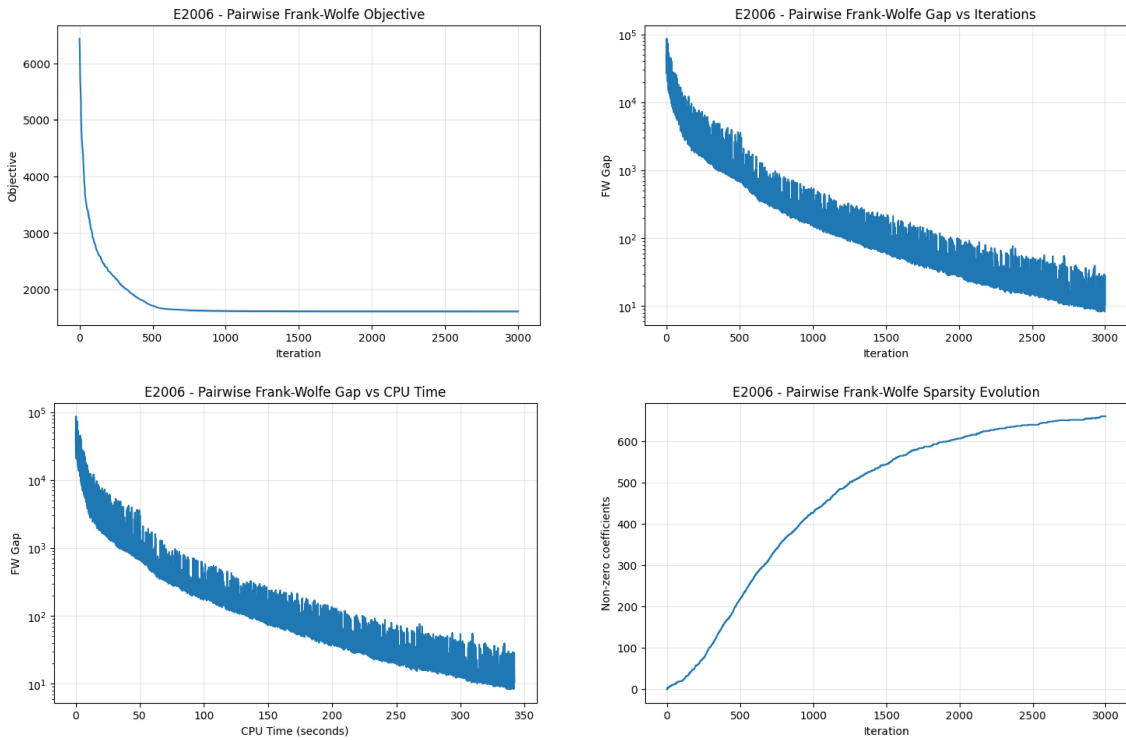


Figure A.3: Pairwise Frank-Wolfe convergence and sparsity results for E2006.

B Classic Frank-Wolfe Convergence Plots

The following figures report the objective value, Frank-Wolfe gap against iterations and CPU time, and sparsity evolution for Classic Frank-Wolfe on the three datasets.

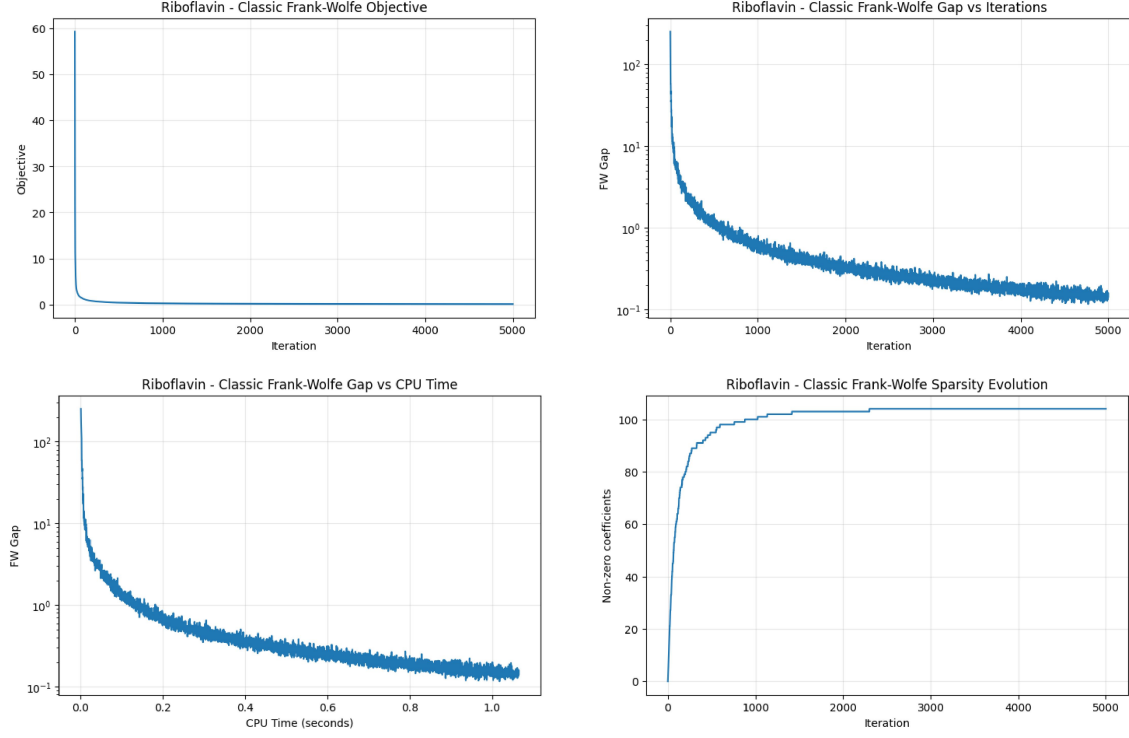


Figure B.1: Classic Frank-Wolfe convergence and sparsity results for Riboflavin.

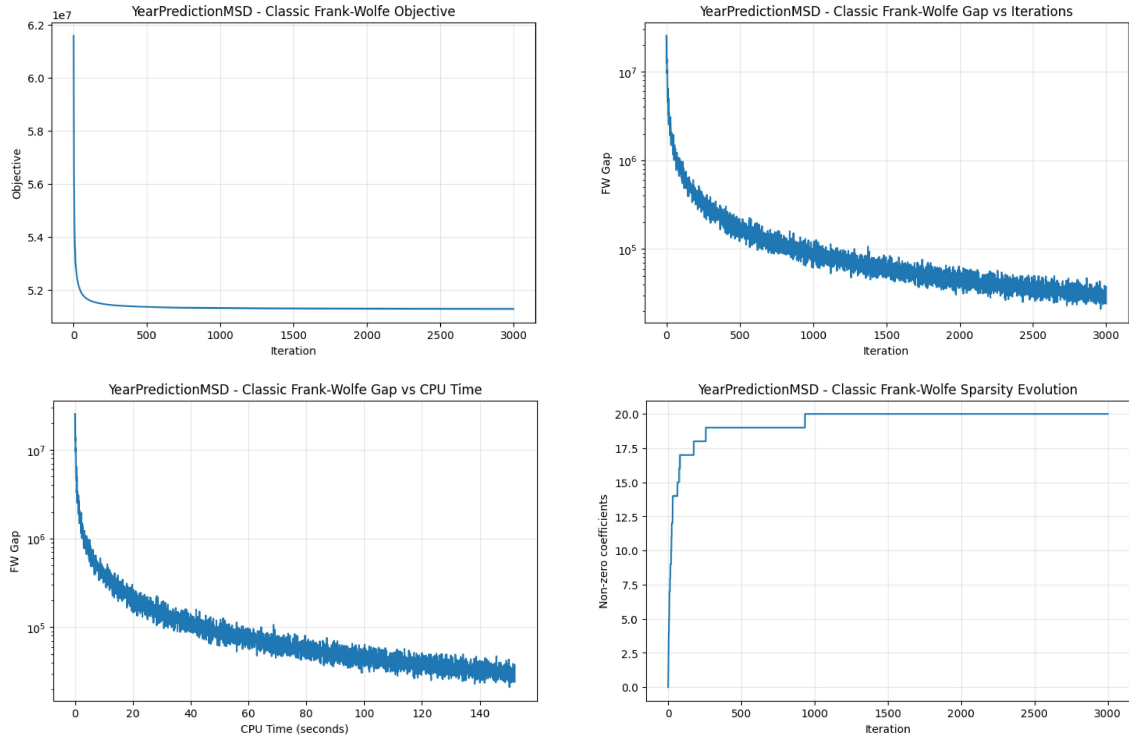


Figure B.2: Classic Frank-Wolfe convergence and sparsity results for YearPredictionMSD.

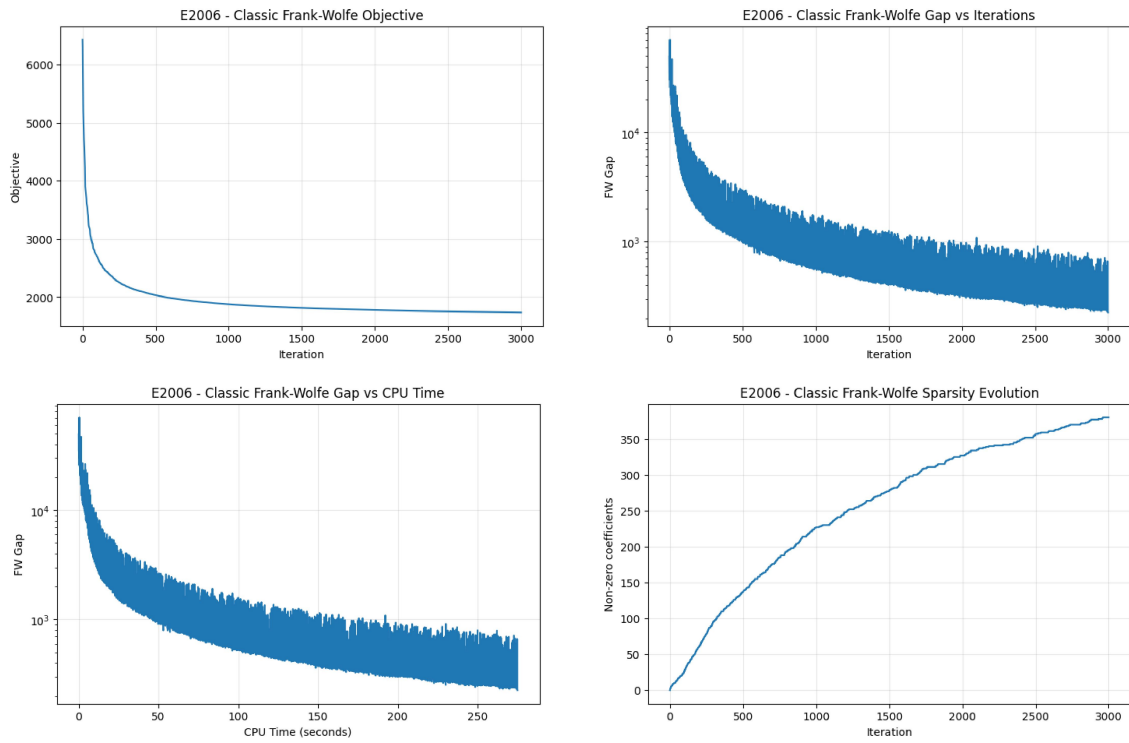


Figure B.3: Classic Frank-Wolfe convergence and sparsity results for E2006.

C Projected Gradient Convergence Plots

The following figures report the objective value, common Frank-Wolfe optimality certificate against iterations and CPU time, and sparsity evolution for Projected Gradient on the three datasets.

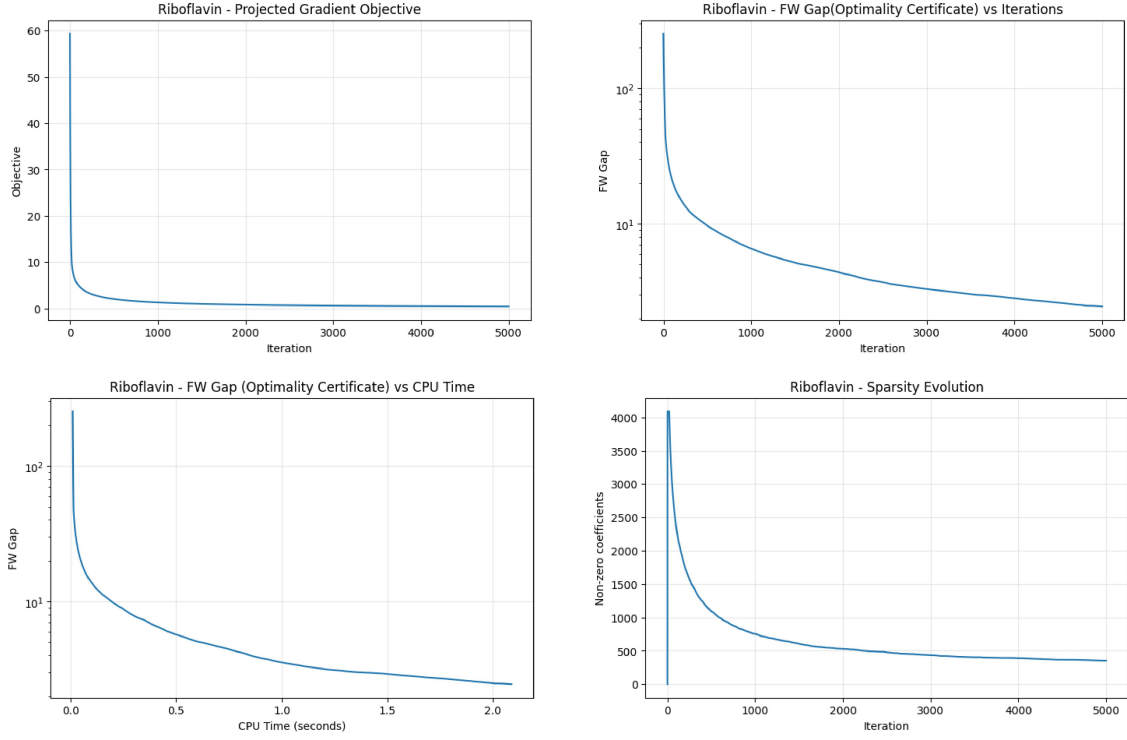


Figure C.1: Projected Gradient convergence and sparsity results for Riboflavin. The FW gap is reported as a common optimality certificate.

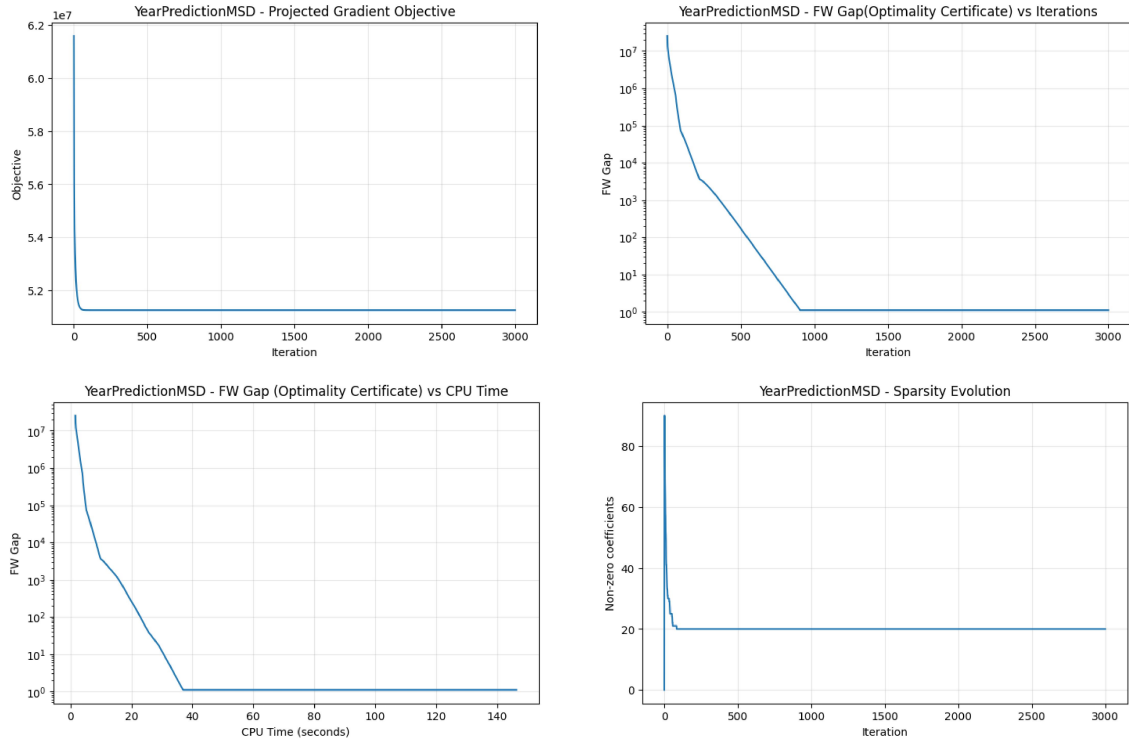


Figure C.2: Projected Gradient convergence and sparsity results for YearPredictionMSD. The FW gap is reported as a common optimality certificate.

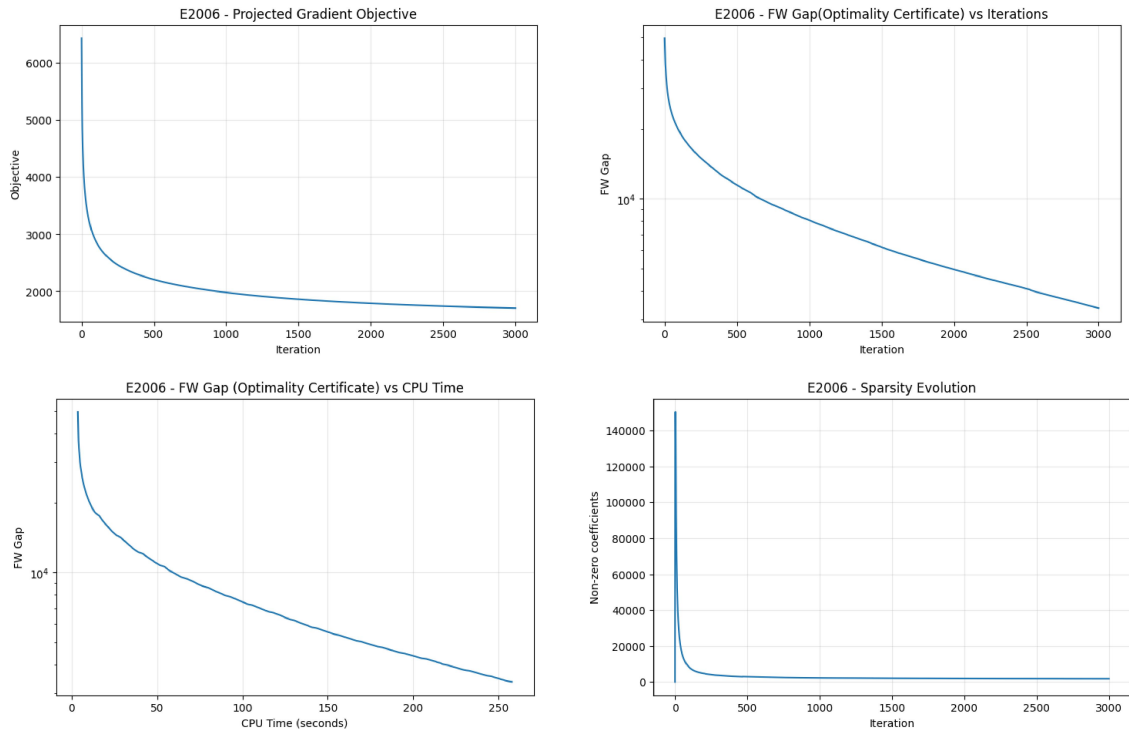


Figure C.3: Projected Gradient convergence and sparsity results for E2006. The FW gap is reported as a common optimality certificate.